

Lab exercises

Part 06 — 🧑 AI Agents · 06-ai-agents/lab/EXERCISES.md

Lab — AI Agents

Work top to bottom. Each exercise has a **stated deliverable**; if you can't produce it, you haven't finished. No API keys are required — everything runs offline against the code in `../code/` or against your own reasoning on paper.

1. Extend the ReAct loop with a new tool and a new failure (1h)

Open `../code/react_loop_from_scratch.py`.

- **1a.** Add a fourth tool, `unit_convert(value: float, from_unit: str, to_unit: str)`, supporting at least `km<->miles` and `celsius<->fahrenheit`. Register it in `TOOLS`.
- **1b.** Write a new scenario function that chains `calculator` and your new tool toward one final answer (e.g. "Convert 100km to miles, then add 5").
- **1c.** Write a scenario where the model calls `unit_convert` with a schema-valid but semantically wrong unit (e.g. `from_unit="lightyears"`) — a case §2 of `notes/02` calls "wrong but valid arguments." Make the tool raise a clear, recoverable error message, not a raw exception.

Deliverable: the diff (or new function) adding the tool and both scenarios, plus one sentence on why 1c is a different failure category than a hallucinated tool name or malformed JSON.

2. Break the max-iteration guard, on purpose and then fix it (30 min)

- **2a.** In `scenario_max_iterations()`, change `max_iterations` to `10` without extending the script. What happens, and why? (Read `run_agent`'s "script exhausted" branch.)
- **2b.** Now extend the script to 10 identical failing steps instead, keeping `max_iterations=10`. Run it. Which guard fires, and does the explanation it prints actually help someone debugging this in production at 2am?
- **2c.** Propose one additional guard condition beyond a step count (e.g. "same action + same arguments repeated 3 times in a row") and describe, in pseudocode, where in `run_agent` you'd add the check.

Deliverable: answers to 2a-2b (2-3 sentences each) and the pseudocode for 2c.

3. The compounding-error math, applied to a real design (1h)

Run `python code/compounding_error_simulator.py`.

- **3a.** You are designing an agent that reads a 15-page contract, extracts 8 key terms, cross-checks them against a policy document, and writes a summary — call it 12 logical steps. If each step is 92% reliable,

what is the end-to-end success rate? Use the script's `analytic_success` function or compute it by hand.

- **3b.** Your team proposes adding a "verifier" LLM call after each extraction step that catches errors 60% of the time. Recompute end-to-end success with that recovery rate. Is it enough? What recovery rate *would* get you to 90% end-to-end?
- **3c.** Propose a redesign that reduces the step count instead of improving per-step accuracy (e.g. combining several extraction calls into one structured- output call). Estimate the new N and recompute.

Deliverable: three numbers (3a, 3b's two variants, 3c) and a one-paragraph recommendation citing which lever (N or p) you'd pull first and why.

4. Planning strategy selection (45 min)

For each task below, name the planning strategy from `notes/02 §1` you'd use, and defend it in ≤ 4 sentences using latency, failure surface, and auditability:

1. "Book me a flight and hotel for a trip to Tokyo next month, staying under \$2000."
2. "Solve this logic puzzle: [a constraint-satisfaction puzzle with several clues]."
3. "Read these five support tickets and tell me if there's a common root cause."
4. "Refactor this 3,000-line module to use the new API, then run the tests, then fix what breaks, repeating until green."

Check: at least one answer should be "plain ReAct, no explicit planning" with a real reason, not a default. At least one should reject Tree of Thoughts even though the task looks search-like, if a cheaper strategy actually fits better — or defend ToT if it doesn't.

5. Trajectory evaluation, by hand (1h)

Take the "1. HAPPY PATH" trajectory printed by `react_loop_from_scratch.py` (calculator, then weather, then final answer).

- **5a.** Write a 4-row rubric (criteria + pass/fail) for grading this trajectory, independent of whether the final answer was correct — e.g. "did each Thought state a reason grounded in the previous Observation, not just restate the goal?"
- **5b.** Now imagine the same final answer was reached, but the model's Thought before calling `get_weather` was "I'll guess it's sunny everywhere" and it called the tool anyway with a fixed city regardless of what the user asked. Score this trajectory against your rubric. Does it pass despite a correct-looking answer?
- **5c.** In one paragraph, explain why 5b is exactly the gap between per-step/outcome grading and trajectory grading described in `notes/02 §4`, and why an eval suite that only checks final answers would never catch it.

Deliverable: the rubric, the 5b score with justification, and the 5c paragraph.

6. Stretch: spot the lethal trifecta (1h)

For each proposed agent below, say whether it qualifies for the lethal trifecta (notes/02 §5), naming which of the three ingredients (untrusted content, private access, exfiltration channel) are present, missing, or borderline:

1. An agent that reads incoming customer emails and drafts (but does not send) replies for a human to approve.
2. The same agent, but now configured to send the replies automatically.
3. An agent that summarizes a user's own uploaded PDF and has no other tools.
4. A coding agent that can browse documentation sites and can also run `git push` to the user's repositories.

Deliverable: a 4-row table (scenario, ingredients present, verdict, one mitigation you'd require before shipping it) — for #1 and #3, explain specifically what would have to change to make them qualify.